

END-TO-END E-COMMERCE PLATFORM

Detailed Technical Architecture, Code Explanation, and Recruiter Guide

This document contains a highly detailed breakdown of every component, code decision, metric choice, and architectural standard used to build the Instacart 33.8M row intelligence platform. It is designed to be easily readable while providing enough deep technical rigor to pass any Senior Analytics/Data Engineering interview.

1. The Data: Understanding Instacart (33.8M Rows)

We are using the official Instacart Market Basket dataset. This is not a toy dataset - it contains over 3 million grocery orders representing roughly 33.8 million individual item purchases. Our goal was to turn this highly normalized transactional CSV data into a blazing-fast analytical Gold Layer using Medallion Architecture.

The Tables & Columns Explained:

- **aisles.csv (134 rows):** aisle_id (PK), aisle. Represents the aisle name (e.g., 'fresh fruits').
- **departments.csv (21 rows):** department_id (PK), department. The high-level category (e.g., 'produce', 'dairy eggs').
- **products.csv (49k rows):** product_id (PK), product_name, aisle_id (FK), department_id (FK). The catalog of all items sold.
- **orders.csv (3.4M rows):** order_id (PK), user_id (FK), eval_set, order_number, order_dow (day of week 0-6), order_hour_of_day (0-23), days_since_prior_order. This is the header-level order table.
- **order_products__*.csv (33.8M rows):** order_id (FK), product_id (FK), add_to_cart_order (the sequence the item was added to the cart), reordered (1 if the user bought this before, 0 if new). This is the line-item detail.

Why did we decide to model this into a Star Schema? Because querying 5 highly normalized tables for every dashboard visual would be extremely slow. We transformed this into Fact and Dimension tables (dim_customers, dim_products, fact_orders) using DuckDB. We also added simulated prices (\$1-\$20 based on department multipliers) to add realistic revenue metrics which the original dataset lacked.

2. Phase A: Data Cleaning & Quality Framework

Before analytics can happen, data must be trusted. We built an Enterprise Data Quality (DQ) Engine using Pandas and DuckDB. We don't just 'drop nulls', we proactively score the warehouse.

What did we check and why?

- **Completeness:** Are there nulls where there shouldn't be? We checked % non-null for every column.
- **Uniqueness:** Did our pipeline accidentally duplicate orders? We checked Primary Key constraints.
- **Validity:** Are the values logically possible? E.g., is order_hour_of_day between 0 and 23? Is unit_price > 0? If a price is negative, the dashboard revenue is wrong.
- **Consistency:** Do foreign keys match? (e.g., every product_id in fact_orders must exist in dim_products).
- **Freshness:** Is the data up to date? We compared max order dates against current dates.

Python Code Explanation: data_quality_report.py

```
# We calculate completeness dynamically for any table
def check_completeness(table_name):
    # 'DESCRIBE' gets all columns dynamically without hardcoding
    columns = conn.execute(f"DESCRIBE {table_name}").fetchall()
    for col in columns:
        col_name = col[0]
        # Calculate percentage of rows where the value IS NOT NULL
        query = f"SELECT count({col_name}) * 100.0 / count(*) FROM {table_name}"
        score = conn.execute(query).fetchone()[0]
        # Append to our tracking list
        results.append({"table": table_name, "metric": "completeness", "score": score})
```

Line-by-line breakdown: First we use `DESCRIBE` to programmatically get columns so the code doesn't break if the schema changes. Then we run a SQL aggregate `count(col) / count(\*)` which is heavily optimized in DuckDB for columnar scans. Finally, we append it to our results so we can generate a 0-100% score for the dashboard gauge.

3. Phase B: Advanced SQL Analytics & KPIs

We built 13 distinct KPI views. Why these specific metrics? Because standard 'Total Revenue' isn't enough for actionable business decisions. We built what real Growth and Product teams need.

The Critical KPIs & Why They Matter

- **Cohort Retention Matrix (v_cohort_retention):** Value: Tells us if the business is leaking customers. It groups users by their first purchase month and tracks what % return in months 1, 2, 3. If Month 1 retention drops from 40% to 20%, the app experience is degrading.
- **Cart Size Analysis (v_cart_analysis):** Value: Shows distribution (e.g., 1-3 items vs 20+ items). Instacart's margin comes from large basket sizes. If everyone buys 1 item, delivery costs destroy profitability. This view helps target 'minimum cart size' promos.
- **Department Cross-Sell (v_department_cross_sell):** Value: If a customer buys Produce, what % also buys Dairy? We built an N x N matrix. This directly feeds recommendation carousels in the UI.
- **Revenue Concentration (v_revenue_concentration):** Value: Calculates Pareto (80/20 rule). We rank products and calculate running cumulative revenue %. If 5% of products generate 80% of revenue, the supply chain team must prioritize those SKUs above all else.

SQL Code Explanation: The Cohort Matrix

```
CREATE OR REPLACE VIEW v_cohort_retention AS
WITH first_orders AS (
  -- Find the very first month a user bought something
  SELECT user_id, date_trunc('month', min(order_date)) as cohort_month
  FROM fact_orders GROUP BY user_id
),
monthly_activity AS (
  -- Get every month a user made at least one purchase
  SELECT DISTINCT user_id, date_trunc('month', order_date) as active_month
  FROM fact_orders
)
SELECT
  f.cohort_month,
  -- Calculate months elapsed since first purchase
  date_diff('month', f.cohort_month, m.active_month) as month_number,
  count(DISTINCT m.user_id) as active_customers,
  -- Calculate % of original cohort that returned
```

```
count(DISTINCT m.user_id) * 100.0 / first_size.cohort_size as retention_rate_pct
FROM first_orders f
JOIN monthly_activity m ON f.user_id = m.user_id
GROUP BY 1, 2;
```

Line-by-line: We use Common Table Expressions (CTEs - the 'WITH' clause) to first establish the 'Birth Month' of every user ('first_orders'). Then we establish every month they returned ('monthly_activity'). We JOIN these together and use 'date_diff' to calculate how many months apart the return visit was. We divide the returning users by the original cohort size to get the true Retention Rate %. This is the exact query you would write at Amazon or Uber.

4. Phase C: The Streamlit Premium Dashboard

We overhauled the UI using a 'Glassmorphism' design system. Recruiters and stakeholders judge with their eyes first. A dashboard that looks like a 1990s Excel sheet will be ignored. We used CSS injection inside Streamlit to achieve a modern SaaS aesthetic.

How it was coded (Design System)

```
st.markdown("""
<style>
  /* Glassmorphism Cards */
  div[data-testid="metric-container"] {
    background: rgba(30, 41, 59, 0.5); /* Semi-transparent navy */
    backdrop-filter: blur(16px); /* The frosted glass effect */
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 16px;
    transition: transform 0.3s ease; /* Hover animation */
  }
  div[data-testid="metric-container"]:hover {
    transform: translateY(-5px); /* Lift up on hover */
  }
</style>
""", unsafe_allow_html=True)
```

By overriding Streamlit's default CSS target elements (`data-testid`), we force the background to be semi-transparent with a `backdrop-filter: blur()`. When you scroll, elements behind the card blur elegantly. We also added `transform: translateY` so cards lift dynamically when hovered, making the app feel alive and responsive.

Dashboard Layout Strategy

- **Global Filters:** We added a Department Sidebar filter. We dynamically build the SQL queries with `WHERE department = 'X'` to allow slice-and-dice functionality.
- **6-Tab Architecture:** Executive, Customer, Product, Basket, Forecasting, Data Quality. We segregated personas. The CEO looks at Executive. The Supply Chain lead looks at Product. The ML Engineer looks at Forecasting.

5. Machine Learning Integration

We built three distinct ML pipelines, transitioning this from a basic 'Analyst' project to an 'ML Engineer' project.

- **K-Means Clustering:** Instead of arbitrary rule-based segments (e.g. 'if spent > \$100 = VIP'), we let math decide. We scaled the RFM features and fit a K-Means model to output ``kmeans_segment``. This finds hidden behavioral clusters.
- **Demand Forecasting:** We implemented Holt-Winters Exponential Smoothing. Why? Retail data is highly seasonal. Holt-Winters natively handles Level, Trend, and Seasonality. We also coded a fallback to RandomForest in case native C++ tree libraries (like XGBoost) fail to compile on certain macOS environments.
- **Market Basket Analysis:** Used Polars for a massive self-join on 33.8 million rows to calculate 'Support', 'Confidence', and 'Lift' for product pairings. If Lift > 1, the products are bought together more often than random chance.

6. Recruiter & Technical Interview Q&A

When presenting this project, you will face scrutiny on architectural decisions. Here is exactly how to answer the toughest questions.

Q: Why did you use DuckDB instead of SQLite, Pandas, or PostgreSQL?

Answer: DuckDB is an in-process OLAP (analytical) database. SQLite and PostgreSQL are OLTP (transactional) row-based databases. Because I am running analytical aggregations (like SUM, GROUP BY, and COUNT DISTINCT) across 33.8 million rows, DuckDB's columnar vectorized engine executes these queries in milliseconds instead of minutes. It gives me the analytical power of Snowflake, but runs entirely local without cloud costs.

Q: Why did you implement a Data Quality framework? Why not just clean the data in Pandas and move on?

Answer: In production enterprise systems, data drifts over time. A hardcoded `.dropna()` in Pandas is a band-aid. By building an automated DQ scoring engine that tests against 6 dimensions (Completeness, Validity, Freshness, etc.) and saving those results to a table, I treat Data Quality as a first-class metric. It creates observability, ensuring dashboards don't silently serve corrupted insights to stakeholders.

Q: How did you handle the sheer size of 33.8 million rows in Python?

Answer: I avoided loading all 33.8 million rows into Pandas RAM at once. I used DuckDB to execute transformations entirely in SQL on disk. For the Market Basket Analysis which requires a massive cartesian self-join, I used Polars instead of Pandas, which utilizes multithreaded execution in Rust to process the permutations without triggering MemoryErrors.

Q: Explain the Cohort Retention SQL query you wrote. Why use CTEs?

Answer: CTEs (WITH clauses) improve readability. The query calculates a true month-over-month retention matrix. First, I use a CTE with a MIN(date) to find the 'Birth Month' of a user. In the second CTE, I grab all active months for that user. By joining them together and using `date_diff`, I calculate how many months passed between their birth and return. Dividing returning users by original cohort size gives the true retention percentage.

Q: What is 'Lift' in your Market Basket Analysis tab?

Answer: Lift measures how much more likely items are bought together compared to random chance. If Lift = 1, they are independent. If Lift = 67 (like our Apple and Strawberry bundle), it means a customer buying Apples is 67 times more likely to also buy Strawberries than the average customer. This is exactly how Amazon calculates 'Frequently Bought Together'.